Enseignante : M.Karim Ghammam	Virtualization et Cloud **TP1**	
	Virtualisation légère / Containers Docker	

Remarque : pour Ubuntu 26.04, vérifiez la disponibilité des paquets dans les dépôts de la version installée. Le nom du paquet Compose peut varier.

Objectif général : installer Docker depuis les paquets Ubuntu, manipuler des images et conteneurs, déployer un serveur Web, construire une image personnalisée, utiliser des volumes, des réseaux et Docker Compose.

1. Objectifs pédagogiques

- Installer et vérifier Docker à partir du paquet Ubuntu docker.io.
- Distinguer une image Docker d'un conteneur.
- Déployer Nginx et accéder au service Web depuis un navigateur.
- Construire une image personnalisée avec un Dockerfile.
- Conserver des données avec un volume Docker.
- Créer un réseau bridge et tester la communication entre conteneurs.
- Déployer plusieurs services avec Docker Compose.

2. Préparation de la machine virtuelle

- Processeur : 2 vCPU minimum.
- Mémoire : 4 Go recommandés.
- Disque : 25 Go minimum.
- Réseau VMware : NAT pour un accès Internet simple, ou Bridged.
- Ouvrir un terminal Ubuntu avec un compte autorisé à utiliser sudo.

Les commandes utilisent sudo docker afin d'éviter de modifier les permissions du démon Docker.

3. Installation de Docker avec les paquets Ubuntu

3.1 Mettre à jour les paquets

```
sudo apt update
sudo apt upgrade -y
```

3.2 Installer et activer Docker

```
sudo apt install -y docker.io
sudo systemctl enable --now docker
```

3.3 Vérifier l'installation

```
docker --version
sudo systemctl status docker
```

sudo usermod -aG docker $USER

reboot

```
sudo docker run --rm hello-world
```

Résultat attendu : hello-world affiche un message confirmant que Docker a téléchargé et exécuté une image de test.

En cas de problème, vérifier le service avec sudo systemctl status docker et les journaux avec sudo journalctl -u docker --no-pager -n 50.

4. Découvrir les images Docker

Une image est un modèle en lecture seule utilisé pour créer des conteneurs. Téléchargez les images :

```
sudo docker pull ubuntu:24.04
sudo docker pull nginx:latest
sudo docker images
```

Travail demandé : relever le nom, le tag et la taille des images. Expliquer la différence entre une image et un conteneur.

5. Créer et gérer un conteneur Ubuntu

5.1 Créer un conteneur interactif

```
sudo docker run -it --name mon_ubuntu ubuntu:24.04 bash
```

Dans le conteneur, exécuter :

```
cat /etc/os-release
hostname
apt update
apt install -y curl iputils-ping
exit
```

5.2 Observer et redémarrer le conteneur

```
sudo docker ps
sudo docker ps -a
sudo docker start -ai mon_ubuntu
sudo docker stop mon_ubuntu
sudo docker rm mon_ubuntu
```

Questions : quelle différence y a-t-il entre docker ps et docker ps -a ? Que font start, stop et rm ?

6. Déployer un serveur Web Nginx

6.1 Lancer Nginx

```
sudo docker run -d --name mon_nginx -p 8080:80 nginx:latest
sudo docker ps
sudo docker logs mon_nginx
curl http://localhost:8080
```

Ouvrir http://localhost:8080 dans le navigateur. Le paramètre -p 8080:80 redirige le port 8080 de la machine vers le port 80 du conteneur.

6.2 Tester le cycle de vie

```
sudo docker stop mon_nginx
sudo docker start mon_nginx
sudo docker restart mon_nginx
sudo docker rm -f mon_nginx
```

7. Construire une image personnalisée

7.1 Créer une page Web

```
mkdir -p ~/tp4-docker/site
cd ~/tp4-docker
cat > site/index.html <<'EOF'
<!DOCTYPE html>
<html lang="fr">
<head><meta charset="UTF-8"><title>TP4 Docker</title></head>
<body>
  <h1>Bienvenue dans mon conteneur Docker !</h1>
  <p>Site Web du TP4 Virtualisation et Cloud.</p>
</body>
</html>
EOF
```

7.2 Créer le Dockerfile

```
cat > Dockerfile <<'EOF'
FROM nginx:alpine
COPY site/index.html /usr/share/nginx/html/index.html
EXPOSE 80
EOF
```

7.3 Construire et tester

```
sudo docker build -t tp4-web:1.0 .
sudo docker run -d --name tp4_site -p 8081:80 tp4-web:1.0
curl http://localhost:8081
```

Tester aussi http://localhost:8081 dans le navigateur. Expliquer FROM, COPY et EXPOSE.

8. Persistance des données avec un volume

Un volume conserve les données indépendamment du cycle de vie du conteneur.

```
sudo docker volume create data_volume
sudo docker run --rm -v data_volume:/data ubuntu:24.04 \
  bash -c 'echo "Donnée persistante du TP4" > /data/message.txt'
sudo docker run --rm -v data_volume:/data ubuntu:24.04 \
  cat /data/message.txt
sudo docker volume ls

cd /var/www/html/

sudo docker run -d  --name nginx1  -p 8080:80  -v  .:/usr/share/nginx/html  nginx:latest

http://localhost:8080

http://localhost:80

sudo nano /var/www/html/index.html
```

Résultat attendu : le deuxième conteneur affiche le message créé par le premier. Expliquer pourquoi le fichier est toujours disponible.

9. Réseau Docker personnalisé

9.1 Créer le réseau et les conteneurs

```
sudo docker network create mon_reseau
sudo docker run -dit --name ubuntu1 --network mon_reseau ubuntu:24.04 bash
sudo docker run -dit --name ubuntu2 --network mon_reseau ubuntu:24.04 bash
sudo docker network inspect mon_reseau
```

9.2 Tester la communication

```
sudo docker exec ubuntu1 bash -c 'apt update && apt install -y iputils-ping curl'
sudo docker exec ubuntu1 ping -c 4 ubuntu2
```

9.3 Ajouter un serveur Web

```
sudo docker run -d --name nginx_reseau --network mon_reseau nginx:alpine
sudo docker exec ubuntu1 curl http://nginx_reseau
```

Nettoyage des ressources de cette partie :

```
sudo docker rm -f ubuntu1 ubuntu2 nginx_reseau
sudo docker network rm mon_reseau
```

10. Docker Compose

Vérifier d'abord la disponibilité de Compose v2 :

```
sudo apt install -y docker-compose-v2
sudo docker compose version
```

Si docker-compose-v2 n'est pas disponible dans le dépôt de la version Ubuntu installée, exécuter apt search docker-compose et installer une version compatible Compose v2. La suite utilise docker compose (avec un espace).

10.1 Créer compose.yaml

```
cd ~/tp4-docker
cat > compose.yaml <<'EOF'
services:
  web:
    image: nginx:alpine
    ports:
      - "8083:80"
    volumes:
      - ./site:/usr/share/nginx/html:ro
    networks:
      - tp4_network
    restart: unless-stopped
  ubuntu:
    image: ubuntu:24.04
    command: ["sleep", "infinity"]
    networks:
      - tp4_network
networks:
```

```
  tp4_network:
    driver: bridge
EOF
```

10.2 Démarrer et tester

```
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs
curl http://localhost:8083
sudo docker compose exec ubuntu bash -c 'apt update && apt install -y curl'
sudo docker compose exec ubuntu curl http://web
```

Ouvrir http://localhost:8083 dans le navigateur. Le dossier local site est monté en lecture seule dans le conteneur Web.

10.3 Arrêter les services

```
sudo docker compose down
```

11. Mini-projet de synthèse

Consigne : créer un site Web personnalisé et le déployer avec Docker Compose sur le port 8085.

- Créer une page HTML avec le nom du projet et une courte description.
- Construire une image personnalisée basée sur nginx:alpine avec un Dockerfile.
- Créer compose.yaml avec un service Web accessible sur le port 8085.
- Utiliser un volume ou un montage de dossier pour rendre le contenu modifiable ou persistant selon la solution choisie.
- Définir un réseau Docker et tester la communication entre services si plusieurs services sont utilisés.
- Tester le site avec le navigateur et curl, puis arrêter proprement les services.

Commandes de vérification :

```
sudo docker images
sudo docker ps -a
sudo docker volume ls
sudo docker network ls
sudo docker compose ps
curl -I http://localhost:8085
```

Livrables : Dockerfile, compose.yaml, fichiers HTML, captures d'écran des tests et court compte rendu expliquant les commandes et les résultats.

12. Questions de synthèse

- Quelle est la différence entre une machine virtuelle et un conteneur ?
- Quelle est la différence entre une image et un conteneur Docker ?
- À quoi sert la publication de ports avec -p ?
- Pourquoi utiliser un volume Docker ?
- Quel est l'intérêt d'un réseau Docker personnalisé ?

- Quel est le rôle de compose.yaml et de docker compose up -d ?
- Quelle différence existe entre docker stop, docker rm et docker compose down ?

13. Tableau récapitulatif des commandes

Commande	Rôle
sudo docker images	Lister les images locales
sudo docker ps -a	Lister tous les conteneurs
sudo docker logs NOM	Afficher les journaux d'un conteneur
sudo docker exec -it NOM bash	Ouvrir un shell dans un conteneur actif
sudo docker stop NOM	Arrêter un conteneur
sudo docker rm NOM	Supprimer un conteneur arrêté
sudo docker volume ls	Lister les volumes
sudo docker network ls	Lister les réseaux
sudo docker compose up -d	Démarrer les services Compose
sudo docker compose down	Arrêter et supprimer les ressources Compose du projet

Fin du TP4 — Bon travail !